inalco Institut national des langues et civilisations orientales

Contrôle 1 : Techniques du Web – Corrigé

Master 2 | *Techniques du Web*

CORRIGÉ	
Barème :	**/ 20**

I. Des questions de code (/ 8 points)

1) Question de fichiers et JSON (2 points)

```python
import json
from pathlib import Path

chemin = Path("/home/user/documents/file.json")
mon_objet = {"name": "Alice", "age": 30, "prof": False}

with open(chemin, "w") as f:
    json.dump(mon_objet, f)
```

a) Quel est le type des variables `chemin`, `mon_objet` et `f` ?

- `chemin` : un objet **`Path`** du module `pathlib` (plus précisément `PosixPath` sur Linux/Mac ou `WindowsPath` sur Windows).
- `mon_objet` : un dictionnaire Python (**`dict`**).
- `f` : un objet fichier ouvert en mode écriture texte (**`_io.TextIOWrapper`**). La réponse « objet fichier » (ou « flux de fichier ») est considérée comme correcte.

b) À quoi sert la fonction `dump` ? Donnez sa signature (entrée et sortie).

`json.dump` permet de convertir (sérialiser) un objet Python au format JSON et de l'écrire directement dans un fichier ouvert.
Entrées : l'objet Python à sauvegarder (`mon_objet`) et le fichier ouvert en écriture (`f`), soit `json.dump(obj, fp)`.
Sortie : la fonction ne renvoie rien (**`None`**), elle écrit directement dans le fichier.

c) Quelle fonction permet de charger ce fichier en un objet Python ?

C'est la fonction **`json.load(f)`**, en lui passant le fichier ouvert en lecture (`"r"`).
*Remarque : attention à ne pas confondre avec **`json.loads()`** qui prend en entrée une chaîne de caractères (`str`) contenant du JSON, et non un fichier.*

2) Cherchez l'erreur (Exceptions et logging) (2 points)

```python
import logging

def diviser_par_contenu(chemin_fichier):
    logging.debug("Lecture du fichier")

    try:
        with open(chemin_fichier, "r") as fichier:
            contenu = fichier.read()
            valeur = int(contenu)
            resultat = 100 / valeur
    except ValueError:
        logging.error("Erreur de conversion")

    return resultat
```

a) Que fait ce code ? Quel problème surviendra à l'exécution si le fichier ne contient pas un entier valide ?

Cette fonction ouvre un fichier texte, lit son contenu, le convertit en entier (`int`) et calcule `100 / valeur` avant de retourner le résultat.

Le problème (l'erreur dans le code) : si le fichier contient du texte qui n'est pas un entier (par exemple `"abc"`), la ligne 9 `int(contenu)` plante et lève une `ValueError`. On saute alors directement dans le bloc `except ValueError:` (ligne 11) **sans jamais avoir exécuté la ligne 10** (`resultat = 100 / valeur`).
Le message d'erreur est bien affiché par `logging.error`, mais la fonction ne s'arrête pas là : elle continue jusqu'à la ligne 14 `return resultat`. Or la variable `resultat` n'a jamais été créée ! Python plante donc avec une erreur `UnboundLocalError` (*cannot access local variable 'resultat' where it is not associated with a value*).
(Remarque : si le fichier contient `0`, une `ZeroDivisionError` non gérée sera levée, et si le fichier n'existe pas, une `FileNotFoundError` sera levée).

b) Que faudrait-il ajouter (ligne 13) pour que, quoi qu'il en soit, le programme imprime à l'écran le chemin du fichier ?

Il faut ajouter un bloc `finally:` juste après le `except` :

```
    finally:
        print(chemin_fichier)
```

Le code placé dans un bloc `finally` est toujours exécuté, qu'il y ait eu une erreur ou non (et même si une exception non rattrapée fait planter la fonction).

c) Expliquez la différence entre `logging.debug` et `logging.error`.

Ce sont deux niveaux de gravité différents du journal (*logger*) :

— `logging.debug` (niveau 10) sert à afficher des messages de suivi très détaillés pendant qu'on développe ou débogue le programme. Par défaut en Python, le seuil d'affichage est réglé sur `WARNING` (niveau 30), donc `logging.debug` n'affiche rien à l'écran si on ne l'active pas explicitement.

— `logging.error` (niveau 40) sert à signaler un vrai problème qui a empêché une opération de fonctionner normalement. Comme son niveau est supérieur à 30, il s'affiche par défaut dans la console.

3) Extraction Web (2 points)

```
1  import urllib.request
2  from bs4 import BeautifulSoup
3
4  url = "https://www.exemple.com/profils"
5  requete = urllib.request.Request(url, headers={"User-Agent": "Etudiant INALCO"})
6
7  with urllib.request.urlopen(requete) as page:
8      html = page.read()
9
10 soup = BeautifulSoup(html, "html.parser")
11 balises_age = soup.find_all("age")
12
13 for balise in balises_age:
14     texte = balise.get_text()
15     print(texte)
```

a) Que fait ce code ? Quel est le nom donné à cette technique ?

Ce script se connecte à la page web `https://www.exemple.com/profils` (en se présentant avec le `User-Agent "Etudiant INALCO"`), télécharge le code HTML de la page, puis utilise `BeautifulSoup` pour chercher toutes les balises `<age>` et afficher leur contenu.
Cette technique s'appelle le **Web Scraping** (ou moissonnage du web).

b) À quoi sert précisément la méthode `get_text()` dans la boucle finale ?

Quand `find_all("age")` récupère un élément, on obtient l'objet balise complet (par exemple `<age>25</age>` ou `<age><b>25</b> ans</age>`).
La méthode **`get_text()`** permet de nettoyer toutes les balises HTML pour ne récupérer que le texte brut situé à l'intérieur (ici la chaîne `"25"` ou `"25 ans"`).

4) Pythonista (2 points)

```
1 def formater_mots(separateur, *mots):
2     elements = []
3     for i in range(len(mots)):
4         elements.append(str(i) + "-" + mots[i].upper())
5     return separateur.join(elements)
6
7 mots_cles = ["python", "web", "inalco"]
8 resultat = formater_mots(" ~ ", *mots_cles)
9 print(resultat)
```

a) Exécutez le code dans votre tête. Que vaut `resultat` ?

À chaque tour de boucle, on colle l'indice `i`, un tiret, et le mot en majuscules : `"0-PYTHON"`, puis `"1-WEB"`, puis `"2-INALCO"`. Ensuite on les relie avec `" ~ "`.
La variable `resultat` vaut donc exactement :
`"0-PYTHON ~ 1-WEB ~ 2-INALCO"`

b) Expliquez le rôle de `join` et de l'étoile dans `*mots` et `*mots_cles`.

- **`separateur.join(elements)`** : colle toutes les chaînes de la liste `elements` en une seule chaîne de caractères, en glissant `separateur` (`" ~ "`) entre chaque élément.
- **`*mots` dans la définition `def formater_mots(separateur, *mots)`** : permet à la fonction d'accepter un nombre quelconque d'arguments après `separateur`. Python les regroupe automatiquement dans un **`tuple`** appelé `mots` (ici `('python', 'web', 'inalco')`).
- **`*mots_cles` lors de l'appel** : fait l'inverse (dépaquetage / *unpacking*). Au lieu de passer la liste d'un bloc, l'étoile « déballe » la liste pour passer ses 3 éléments comme 3 arguments séparés : `formater_mots(" ~ ", "python", "web", "inalco")`.

c) Comment pourrait-on réécrire les lignes 3 et 4 pour être plus efficace et éviter d'écrire `mots[i]` ?

En Python, quand on a besoin à la fois de l'indice et de la valeur dans une boucle, on utilise **`enumerate()`** (et une *f-string* pour éviter les `+` et `str(i)`) :

```
for i, mot in enumerate(mots):
    elements.append(f"{i}-{mot.upper()}")
```

On peut même remplacer toute la fonction en une seule ligne avec une compréhension :

```
return separateur.join(f"{i}-{mot.upper()}" for i, mot in enumerate(mots))
```

II. Data Science à choix multiples (/ 6 points)

Rappel du dataset `df` :

Pays	Marque	Prix 2025 (EUR)	Prix 2026 (EUR)	Prix fournisseur (EUR)
France	Marlboro	12.50	13.00	1.50
France	Camel	12.00	12.50	1.40
Suisse	Marlboro	10.20	10.50	1.50
Suisse	Camel	9.90	10.20	1.40
Allemagne	Marlboro	8.40	8.70	1.50
Allemagne	Camel	8.20	8.50	1.40

Question 1 : Quelles instructions permettent d'obtenir les dimensions exactes du dataset (le nombre de lignes et de colonnes) ?

Options : a) `df.size` *b)* `df.shape` *c)* `len(df)` *d)* `df.info()`

Réponse attendue : b) `df.shape`
`df.shape` renvoie directement le tuple `(6, 5)`, soit 6 lignes et 5 colonnes.
Pourquoi pas les autres : `df.size` renvoie `30` (le nombre total de cases 6×5), et `len(df)` ne donne que le nombre de lignes (`6`). Quant à `df.info()`, elle ne renvoie rien (`None`), mais affiche dans la console un résumé où figurent `6 entries` et `total 5 columns`.

Question 2 : Quelles commandes permettent d'extraire le prix minimum des cigarettes pour l'année 2025 ?

Options : a) `df.min('Prix_2026')` *b)* `df['Prix_2025'].min()` *c)* `df['Prix_2025'].max()` *d)* `min(df['Prix_2025'])`

Réponses attendues : b) `df['Prix_2025'].min()` **et d)** `min(df['Prix_2025'])`
Les deux commandes fonctionnent en Python et renvoient `8.2` (8,20 EUR, Camel en Allemagne) :
- **b)** utilise la méthode Pandas `.min()` sur la colonne `Prix_2025`.
- **d)** passe la colonne Pandas à la fonction native `min()` de Python, ce qui marche aussi très bien.
- **a)** regarde la mauvaise année et plante avec une `ValueError` (car l'argument de `df.min()` est un axe `0` ou `1`, pas un nom de colonne), et **c)** calcule le maximum (`12.5`).

Question 3 : Vous souhaitez calculer l'évolution (ou le taux d'évolution) du prix entre 2025 et 2026. Quelle(s) ligne(s) font cela ?

Options : a) `df['Evolution'] = df['Prix_2026'] / df['Prix_2025']`
b) `df['Evolution'] = (df['Prix_2026'] - df['Prix_2025']) / df['Prix_2025']`
c) `df['Prix_2026'] - df['Prix_2025']` *d)* `df['Prix_2025'] / df['Prix_2026']`

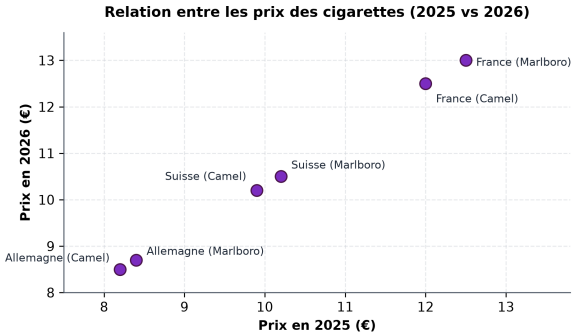
Réponse attendue : b) *(les réponses* ***c)*** *et* ***a)*** *sont également considérées comme correctes selon la justification).*
- **b)** calcule le **taux d'évolution** relatif $\frac{\text{Prix}_{2026} - \text{Prix}_{2025}}{\text{Prix}_{2025}}$ (par exemple `0.04` soit +4% pour Marlboro en France) et le stocke dans une nouvelle colonne `Evolution`.
- **c)** calcule l'**évolution du prix** sans l'enregistrer dans une colonne.
- **a)** calcule le coefficient multiplicateur ($\approx 1{,}04$).

Question 4 : Que fait exactement la ligne chaînée `df.groupby('Marque')['Prix_2026'].mean()` **?**

Réponses attendues : b) et c)
- **b)** décrit bien le calcul effectué : on regroupe les lignes par marque (`Camel` d'un côté, `Marlboro` de l'autre) et on calcule la moyenne de leurs prix en 2026.
- **c)** décrit précisément le résultat renvoyé par Pandas : un objet de type `Series`

qui a les marques comme index (`Camel`, `Marlboro`) et les moyennes 2026 comme valeurs (`Camel : 10.40`, `Marlboro : 10.733333`).

— **a)** est faux car `df` n'est pas modifié, et **d)** est faux car la moyenne est séparée par marque.



Question 5 : Quelle commande Matplotlib permet de générer ce graphique ?

Réponse attendue : b) `plt.scatter(df['Prix_2025'], df['Prix_2026'])`
C'est un nuage de points (*scatter plot*) avec le prix 2025 sur l'axe X et le prix 2026 sur l'axe Y. (`plt.hist` fait un histogramme, `plt.plot` trace une courbe reliant les points, et `plt.bar` un diagramme en barres).

Question 6 : À quoi peut concrètement servir ce type de graphique (nuage de points) dans notre analyse ?

Réponses attendues : b) et c)

— **b)** Il permet de voir tout de suite qu'il y a une très forte corrélation linéaire positive entre le prix de 2025 et celui de 2026 (les points sont alignés sur une droite croissante : plus c'était cher en 2025, plus c'est cher en 2026).

— **c)** Il permet aussi de repérer d'un coup d'œil une valeur aberrante (*outlier*), par exemple si un pays avait doublé ses taxes en 2026, son point sortirait complètement de l'alignement.

III. Étude de mots et algorithmique (/ 8 points)

```python
def is_palindrome(substring):
    if len(substring) < 2:
        return True

    for i in range(len(substring) // 2):
        if substring[i] == substring[-(i+1)]:
            return True

    return False

def longestPalindrome(s):
    longest = 0
    longest_substring = ""

    for i in range(len(s) + 1):
        for j in range(i, len(s) + 1):
            substring = s[i:j]

            if is_palindrome(substring):
                if len(substring) > longest:
                    longest = len(substring)
                    longest_substring = substring

    print(longest_substring)
    return longest_substring
```

a) Expliquez à quoi il sert (entrée / sortie).

Entrée : une chaîne de caractères `s` (`str`).
Sortie : une chaîne de caractères (`str`), affichée avec `print` et renvoyée par `return`.
But de l'algorithme : il est censé parcourir toutes les sous-chaînes possibles du mot `s` pour trouver et retourner le plus long sous-mot qui soit un palindrome (un mot qui se lit pareil de gauche à droite et de droite à gauche, comme `"radar"` ou `"carac"`).

b) Essayer l'algorithme avec le mot `"caracas"` en détaillant les principales étapes. Vous pouvez présenter les résultats dans un tableau. Que constatez-vous ?

Pour `s = "caracas"` (longueur 7), la boucle externe fait varier `i` de `0` à `7`, et la boucle interne fait varier `j` de `i` à `7`.
Voici le déroulement complet et exact du code tel qu'il est écrit pour `i = 0` et `i = 1` (pour toutes les valeurs de `j`) :

i	j	substring	Ce que fait is_palindrome(substring)	Résultat	longest	longest_sub.
0	0	`""`	`len < 2` → renvoie `True` (mais `0 > 0` est faux)	`True`	0	`""`
0	1	`"c"`	`len < 2` → renvoie `True` (`1 > 0` : **mise à jour**)	`True`	**1**	`"c"`
0	2	`"ca"`	1er tour : `'c' == 'a'` est faux → fin boucle	`False`	1	`"c"`
0	3	`"car"`	1er tour : `'c' == 'r'` est faux → fin boucle	`False`	1	`"c"`
0	4	`"cara"`	1er tour : `'c' != 'a'` ; 2e tour : `'a' != 'r'`	`False`	1	`"c"`
0	5	`"carac"`	1er tour : `'c' == 'c'` est vrai → `return True` (**mise à jour**)	`True`	**5**	`"carac"`
0	6	`"caraca"`	1er tour : `'c'!='a'` ; 2e : `'a'!='c'` ; 3e : `'r'!='a'`	`False`	5	`"carac"`
0	7	`"caracas"`	1er tour : `'c' != 's'` (continue !) ; **2e tour : `'a' == 'a'` → `True` !**	`True`	**7**	`"caracas"`
1	1	`""`	`len < 2` → `True` (mais `0 > 7` est faux)	`True`	7	`"caracas"`
1	2	`"a"`	`len < 2` → `True` (mais `1 > 7` est faux)	`True`	7	`"caracas"`
1	3	`"ar"`	1er tour : `'a' != 'r'` → fin boucle	`False`	7	`"caracas"`
1	4	`"ara"`	1er tour : `'a' == 'a'` → `True` (vrai palindrome, mais `3 <= 7`)	`True`	7	`"caracas"`
1	5	`"arac"`	1er tour : `'a' != 'c'` ; 2e tour : `'r' != 'a'`	`False`	7	`"caracas"`
1	6	`"araca"`	1er tour : **`'a' == 'a'` → `return True` à tort !** (mais `5 <= 7`)	`True`	7	`"caracas"`
1	7	`"aracas"`	1er tour : `'a'!='s'` ; 2e : `'r'!='a'` ; 3e : `'a'!='c'`	`False`	7	`"caracas"`

Pour la suite (`i = 2` à 7), toutes les sous-chaînes ont une longueur inférieure ou égale à 5, donc plus aucune ne peut dépasser `longest = 7` (on note tout de même que pour `i = 2, j = 7`, la sous-chaîne `"racas"` renvoie elle aussi `True` à tort car au 2e tour de boucle de `is_palindrome`, `'a' == 'a'`).

Constat : À la fin, le programme affiche et renvoie `"caracas"` (longueur 7), alors que `"caracas"` n'est pas du tout un palindrome (à l'envers : `"sacarac"`) ! Le vrai plus long palindrome était `"carac"` (longueur 5).

Regardez bien ce qui s'est passé à l'étape `i = 0, j = 7` sur `"caracas"` :

— Au 1er tour de la boucle de `is_palindrome` (`i = 0`), on compare la 1re lettre `'c'` et la dernière lettre `'s'`. Comme `'c' != 's'`, le `if` n'est pas déclenché... mais la fonction **ne s'arrête pas** et passe au tour suivant !

— Au 2e tour (`i = 1`), elle compare la 2e lettre `substring[1]` (`'a'`) et l'avant-dernière lettre `substring[-2]` (`'a'`). Comme `'a' == 'a'`, elle fait `return True` !

— De la même manière, sur `"araca"` (`i = 1, j = 6`), dès le 1er tour elle voit que la première et la dernière lettre sont un `'a'`, et elle renvoie tout de suite `True` sans même regarder le milieu (`"rac"`).

c) Détection et correction d'erreur : où se situe l'erreur dans le code et comment la corriger ?

L'erreur se trouve dans la fonction `is_palindrome` aux lignes 5 à 9 :

```
    for i in range(len(substring) // 2):
        if substring[i] == substring[-(i+1)]:
            return True
    return False
```

Écrite comme cela, la fonction renvoie `True` dès qu'elle trouve **une seule** paire de lettres symétriques identiques dans le mot. Or, pour être un palindrome, il faut que **toutes** les paires symétriques soient identiques.
Il faut donc faire l'inverse : dès que deux lettres symétriques sont **différentes** (`!=`), on peut s'arrêter et renvoyer `False`. Et ce n'est qu'à la toute fin de la boucle (si aucune différence n'a été trouvée) qu'on renvoie `True` :

```
def is_palindrome(substring):
    if len(substring) < 2:
        return True

    for i in range(len(substring) // 2):
        if substring[i] != substring[-(i+1)]:
            return False

    return True
```

Remarque : en Python, on peut aussi écrire directement `return substring == substring[::-1]`.
Une fois cette correction faite, `longestPalindrome("caracas")` affiche et renvoie bien `"carac"`.

d) Comment pourrait-on écrire les deux lignes (lignes 19–20 ou lignes 21–22) en une seule ligne / plus efficacement ?

Pour écrire les deux affectations (lignes 21 et 22) en une seule ligne :

```
longest, longest_substring = len(substring), substring
```

e) Quelle est la complexité de cet algorithme ?

Si on note N la longueur du mot `s`, la complexité temporelle dans le pire des cas est en $\mathcal{O}(N^3)$ (cubique) :

— Les deux boucles imbriquées `for i` et `for j` génèrent toutes les paires d'indices (i, j) avec $0 \leq i \leq j \leq N$, soit $\frac{(N+1)(N+2)}{2} \approx \frac{N^2}{2}$ sous-chaînes, ce qui donne déjà $\mathcal{O}(N^2)$ tours de boucle.

— À chaque tour de boucle, le découpage `s[i:j]` et la vérification `is_palindrome(substring)` parcourent la sous-chaîne, ce qui prend un temps proportionnel à sa longueur, soit $\mathcal{O}(N)$ dans le pire des cas.

— Au total : $\mathcal{O}(N^2) \times \mathcal{O}(N) = \mathcal{O}(\mathbf{N^3})$.

f) Modifiez le code de la fonction principale pour qu'elle ne cherche plus le palindrome le plus long, mais renvoie tous les palindromes contenus dans le mot pourvu qu'ils fassent plus de 3 lettres.

Au lieu de garder une seule chaîne maximale, on crée une liste (ou un ensemble `set` pour éviter d'éventuels doublons) et on y ajoute chaque sous-chaîne qui vérifie `len(substring) > 3` et `is_palindrome(substring)` :

```
def tousLesPalindromes(s):
    palindromes = []
    for i in range(len(s) + 1):
        for j in range(i, len(s) + 1):
            substring = s[i:j]
            if len(substring) > 3 and is_palindrome(substring):
```

```
                if substring not in palindromes:
                    palindromes.append(substring)
    return palindromes
```

Valeur réelle obtenue en exécutant ce code sur `"caracas"` :

— Avec `is_palindrome` **corrigée** : renvoie `['carac']` (les deux autres vrais palindromes `'ara'` et `'aca'` font exactement 3 lettres, donc ne passent pas le filtre strict `> 3` ; avec `>= 3` on obtient `['carac', 'ara', 'aca']`).

— Avec `is_palindrome` **non corrigée (buggée)** : renvoie `['carac', 'caracas', 'araca', 'racas']`.

g) Que faire pour que le code retourne les mots qui commencent et se terminent par la même lettre, peu importe ce qu'il y a au milieu ?

Il suffit de remarquer que dans `is_palindrome`, le tout premier test de la boucle (`i = 0`) compare justement la première lettre `substring[0]` et la dernière lettre `substring[-1]` !
Pour ne regarder que la première et la dernière lettre sans tester le milieu, on enlève donc la boucle `for` et on teste uniquement `substring[0] == substring[-1]` (pour les sous-mots d'au moins 2 lettres) :

```
def meme_debut_fin(s):
    mots = []
    for i in range(len(s) + 1):
        for j in range(i, len(s) + 1):
            substring = s[i:j]
            if len(substring) >= 2 and substring[0] == substring[-1]:
                mots.append(substring)
    return mots
```

Valeur réelle obtenue en exécutant ce code sur `"caracas"` : renvoie `['carac', 'ara', 'araca', 'aca']`.